

Predictive Analytics System for Website Development Opportunities

~ Prepared By:
George Jibin Verghese





Problem Statement & Objectives

Problem Statement

Many successful local businesses have strong customer ratings and engagement but lack an official website, limiting their digital presence and growth opportunities.

Objectives

- Collect real-world business data using Google Places API.
 - Clean and prepare the dataset for analysis.
 - Explore business patterns through visualization.
 - Develop a machine learning model to predict website ownership.
 - Identify and rank high-potential businesses for website development.
- 
- 



Why Predictive Analytics?

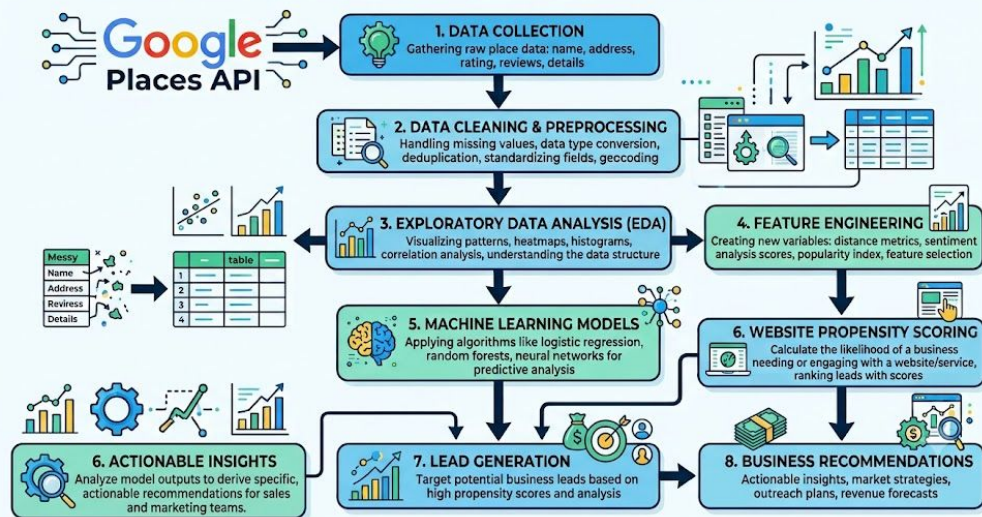
Why is this important?

Traditional lead identification relies on manual searching, which is time-consuming and inefficient.

Predictive Analytics enables organizations to:

- Identify businesses with high growth potential.
 - Prioritize leads using data-driven insights.
 - Reduce manual effort in lead generation.
 - Improve decision-making through machine learning.
- 
- 

DATA-DRIVEN BUSINESS WORKFLOW FOR LEAD GENERATION



Data Workflow

The workflow transforms raw business data into actionable website development opportunities using predictive analytics.

Data Collection

Data Source: Google Places API

Dataset Overview

- 10,000+ business records
- 35 Delhi localities
- 15 business categories
- Automated API-based data collection

Attributes Collected

- | | | |
|-----------------|----------------|-------------------|
| • Business Name | • Review Count | • Business Status |
| • Area | • Website | |
| • Category | • Phone Number | |
| • Rating | • Price Level | |

Key Point

Using live API data provides a realistic and up-to-date dataset for predictive analysis.

```
data_loading.py > search_places
1 import requests
2 import pandas as pd
3 import time
4 import os
5
6 API_KEY = os.environ.get("GOOGLE_PLACES_API_KEY", "I've removed my api key due security reasons")
7
8 MAX_REQUESTS = 950
9
10 AREAS = [
11     "Burari, Delhi", "Karol Bagh, Delhi", "Ashok Vihar, Delhi", "Hauz Khas, Delhi",
12     "Connaught Place, New Delhi", "Pitampura, Delhi", "Rajendra Nagar, Delhi",
13     "Metaji Subhash Place, Delhi", "Rohini, Delhi", "Dwarka, Delhi",
14     "Lajpat Nagar, Delhi", "Saket, Delhi", "Vasant Kunj, Delhi", "Janakpuri, Delhi",
15     "Rajouri Garden, Delhi", "Paharganj, Delhi", "Chandni Chowk, Delhi",
16     "Model Town, Delhi", "Shahdara, Delhi", "Mayur Vihar, Delhi",
17     "Preet Vihar, Delhi", "Laxmi Nagar, Delhi", "Kalkaji, Delhi",
18     "Greater Kailash, Delhi", "South Extension, Delhi", "Malviya Nagar, Delhi",
19     "Munirka, Delhi", "Vikasপুরी, Delhi", "Uttam Nagar, Delhi", "Tilak Nagar, Delhi",
20     "Punjabi Bagh, Delhi", "Shalimar Bagh, Delhi", "Civil Lines, Delhi",
21     "Kamla Nagar, Delhi", "Mukherjee Nagar, Delhi", "Patel Nagar, Delhi",
22     "Moti Nagar, Delhi", "Okhla, Delhi", "Nehru Place, Delhi", "Mayapuri, Delhi",
23 ]
24
25 CATEGORIES = [
26     "restaurants", "cafes", "salons", "gyms", "clothing stores",
27     "grocery stores", "electronics shops", "clinics", "bakeries",
28     "pharmacies", "tutoring centers", "furniture stores", "mobile shops",
29     "jewellery shops", "hardware stores",
30 ]
31
32 OUTPUT_FILE = "delhi_business_raw.csv"
33
34 def search_places(query, api_key):
35     """Calls Google Places API (New) - Text Search. Returns a list of places."""
36     url = "https://places.googleapis.com/v1/places:searchText"
37
38     headers = {
39         "Content-Type": "application/json",
```

Data Preprocessing & Feature Engineering

Data Cleaning

- Removed duplicate records
- Handled missing values
- Standardized text values
- Processed business status
- Verified dataset quality

Engineered Features

- Has Website
- Has Phone
- Review Outlier
- Number of Business Types
- Cleaned Price Level

Outcome

A clean and machine-learning-ready dataset suitable for predictive modelling.

DATA CLEANING & PREPROCESSING

=====

```
df=pd.read_csv("delhi_business_raw.csv")
print(f"\nRaw shape: {df.shape}")
print(f"Columns : {list(df.columns)}")
df.info()
```

DROPPING THE DUPLICATE DATA

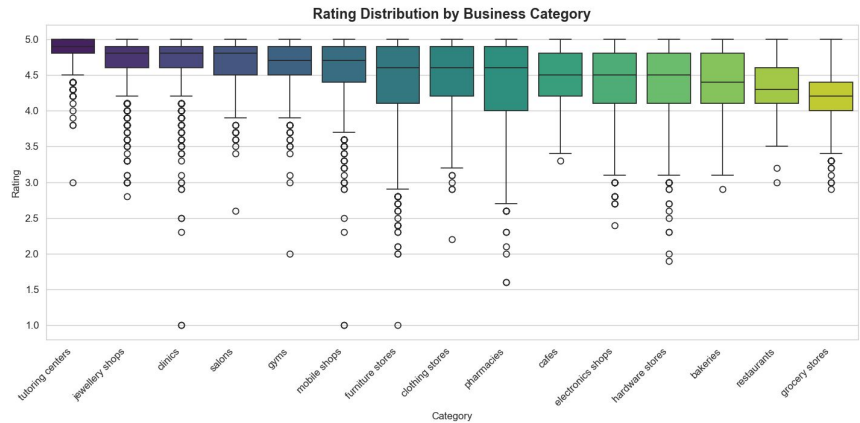
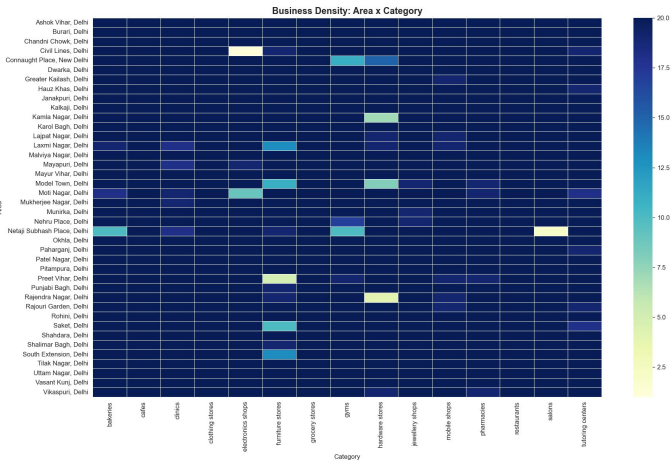
```
before = len(df)
df = df.drop_duplicates(subset=["name", "address"], keep="first")
print(f"\nRemoved {before - len(df)} duplicate business listings")
```

Removed 1781 duplicate business listings

HANDLE MISSING RATINGS / REVIEW COUNTS

```
# A business with no rating/review data can't help us predict rating, and it's a small % of rows (~0.3%),
# so I dropped those rows rather than invent fake ratings.
before = len(df)
df = df.dropna(subset=["rating", "review_count"])
print(f"Dropped {before - len(df)} rows missing rating/review count")
```

EDA & Visualization



Key Insights

- Most businesses maintain ratings above **4.0**.
- Businesses with websites generally receive higher customer engagement.
- Some business categories have significantly lower website adoption.
- Review count shows a positive relationship with business popularity.

Purpose

EDA helped uncover patterns and guided feature selection for model development.

Machine Learning Models



Machine Learning Models Used:

Logistic Regression

- A supervised classification algorithm.
- Predicts the probability of a business having an official website.
- Serves as the baseline model for comparison.
- Easy to interpret and computationally efficient.

Random Forest

- An ensemble learning algorithm.
- Builds multiple decision trees using different subsets of the data.
- Combines predictions from all trees through majority voting.
- Reduces overfitting and improves prediction accuracy.

Why Two Models?

Using multiple models allows performance comparison and helps select the most suitable algorithm for the business problem.

```
> FEATURES_NUM = ["review_count", "rating", "num_types", "price_level_clean"]
FEATURES_CAT = ["area", "category_searched"]
TARGET = "has_website"
X = df[FEATURES_NUM + FEATURES_CAT]
y = df[TARGET]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
print(f"\nTrain size: {len(X_train)}, Test size: {len(X_test)}")

preprocessor = ColumnTransformer(transformers=[("cat", OneHotEncoder(handle_unknown="ignore"), FEATURES_CAT), ("num", Standar

[10]

...
Train size: 9427, Test size: 2357

MODEL 1: LOGISTIC REGRESSION (baseline)

log_pipeline = Pipeline([("prep", preprocessor), ("model", LogisticRegression(max_iter=1000, class_weight="balanced"))])
log_pipeline.fit(X_train, y_train)
log_pred = log_pipeline.predict(X_test)
log_proba = log_pipeline.predict_proba(X_test)[: , 1]

[11]

MODEL 2: RANDOM FOREST

rf_pipeline = Pipeline([("prep", preprocessor), ("model", RandomForestClassifier(n_estimators=300, max_depth=12, random_state=42))])
rf_pipeline.fit(X_train, y_train)
rf_pred = rf_pipeline.predict(X_test)
rf_proba = rf_pipeline.predict_proba(X_test)[: , 1]

[12]
```

Machine Learning Models

Prediction Objective

Predict whether a business is likely to have an **official website** based on its characteristics.

Logistic Regression (Baseline)

- Simple binary classification model.
- Predicts the probability of a business having a website.
- Used as a benchmark for comparison.

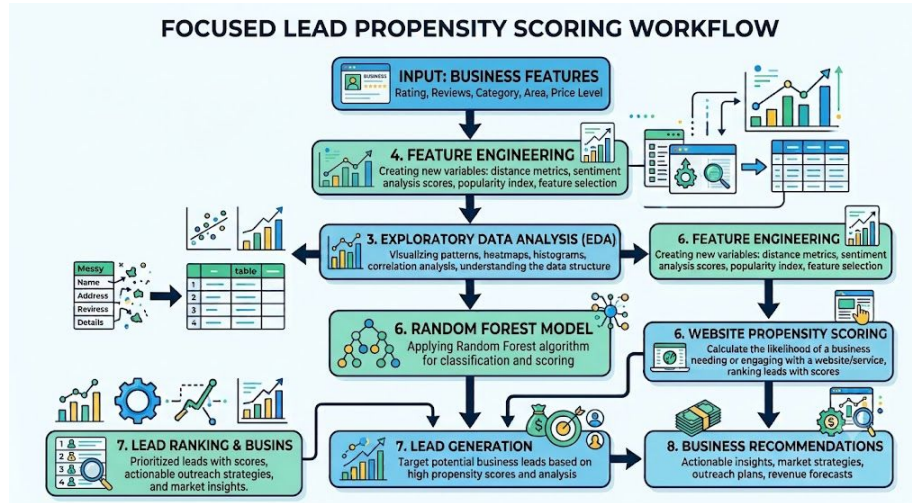
Random Forest (Final Model)

- Ensemble model consisting of **300 decision trees**.
- Each tree makes a prediction using different subsets of the data.
- Final prediction is based on the majority vote of all trees.
- Captures complex relationships between business features.

Why Random Forest?

- Higher predictive accuracy.
- Handles complex feature interactions.
- More robust and less prone to overfitting than a single decision tree.

How the Prediction Works ?



Key Idea

The model learns patterns from businesses that already have websites and assigns a **Website Propensity Score** to every business. Businesses with a **high score but no website** are identified as high-priority website development leads.

Model Performance

Model Results

Accuracy

- 70% of businesses were classified correctly.

ROC-AUC

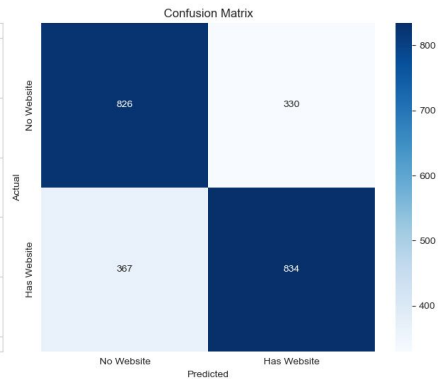
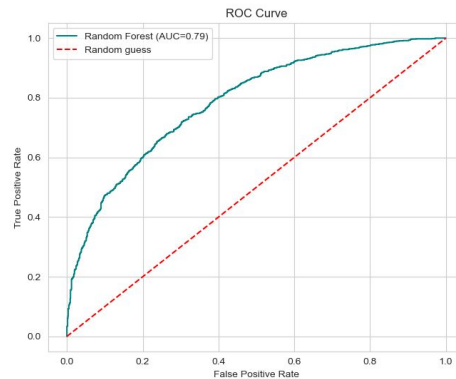
- 0.77, indicating good ability to rank businesses by website propensity.

Confusion Matrix

- Shows correct and incorrect classifications.
- Helps evaluate where the model performs well and where misclassifications occur.

Why ROC-AUC Matters

Since the goal is to **rank businesses** rather than simply classify them, **ROC-AUC is a more meaningful performance metric than accuracy.**



Website Propensity Scoring & Lead Generation



Website Propensity Scoring & Lead Generation

Website Propensity Score

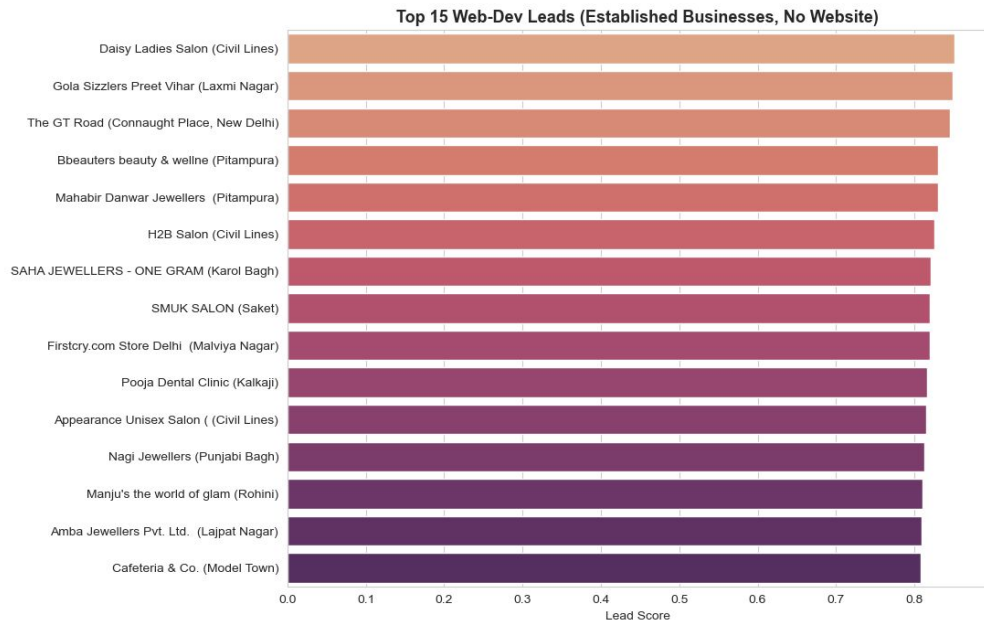
The trained Random Forest model assigns each business a **Website Propensity Score**, representing how closely it resembles businesses that already have an official website.

High-Priority Leads

Businesses with:

- High Ratings
- High Review Counts
- Strong Business Presence
- No Official Website

These businesses were identified as ideal prospects for website development services.





Business Insights & Recommendations

Key Insights

- More than half of the analyzed businesses do not have an official website.
- Review count is the strongest predictor of website ownership.
- Hardware stores, pharmacies, and mobile shops present significant digital opportunities.
- Predictive analytics enables efficient prioritization of website development leads.

Recommendations

- Target businesses with high website propensity scores.
 - Offer customized website development packages.
 - Focus digital marketing efforts on high opportunity business categories.
 - Continuously retrain the model using updated API data.
- 



Conclusion & Future Scope

Conclusion:

This project successfully developed an end-to-end predictive analytics system for identifying businesses with high website development potential.

By combining Google Places API, data preprocessing, exploratory analysis, and machine learning, the system generated actionable business insights and prioritized high-value leads.

Future Scope:

- Real-time Lead Recommendation System
 - Integration with CRM Platforms
 - Automated API Data Collection
- 

Thank You!!!

~ Prepared By:
George Jibin Verghese

